

Price to pay

A high end commercial game engine license costs anywhere between \$ 10,000 to the number running in millions with number of licensees reaching dozens of companies

Fast Physics

Need For Speed on the Playstation ran its physics at 100 Hz as compared to Forza Motorsport 2 running its physics at 360 Hz

How stuff works

part. Game physics helps us deal with things such as gravity, inertia, velocity changes, collisions, slopes and walls.

How real a game feels, depends on the physics of the game. It lies in the game creator to cleverly slip real physics into the game without spoiling the fun part. Once your character is modelled and placed into the game world, it has to interact with the other objects following certain principles of game physics.

Making your characters follow real world physics can be quite unrealistic in the game world. For example, in a racing game such as *Motogp*, after a crash your "Avatar" should not be in a condition to ride the bike according to real physics. However, this cannot be included in the game, as the players expect to have fun.

However, some times including real world physics can make the game more interesting. Games such as *Gran Turismo* and *Need for Speed* gives the player the actual experience of riding a car using real world physics. The braking system employed into the cars of the game are similar to the ones in the real world.

Sound and audio

Blending sound and audio into animation is not as easy as it seems to be. There is a huge difference in the sounds you hear with the source just in front of you and the source in another room. (You can notice it games like *Call of duty: Modern warfare* and *Counter strike*). Sounds in game world follow principles of physics like occlusion and obstruction. Both of them are similar in terms of having an obstacle in between the player and the sound source. Occlusion occurs when the obstacle is encompassing, otherwise it's Obstruction. With these principles in action, there is a huge difference in the sound you hear underwater or in a long corridor.

All game engines support adding sound and audio properties to your game. Options for sound mixing and positioning are available in all major game engines. For example, in Unity, you can change the pitch of each audio clip, stream audios directly from the

web, place positional sound sources anywhere in the scene and thus have an impressive audio experience.

Game networking

LAN play when introduced, gave a new dimension to gaming. It merged the concept of socialisation along with gaming. It's also essential to include networking in your game. LAN play increases the longevity of the game as you always have a new contender to combat. May be that's the small little secret of *Counter Strike* that still rules the college hostels in India. To include LAN play, understanding the concepts of internet operation is important.

Multi-player gaming can be done using two architectures: Peer-to-peer and client/server. Peer-to-peer is where a game is set up on two machines. Each individual machine follows its game, accepts input from the other machine and incorporates updates in its game, frame-by-frame. Up to two players can only be supported in peer-to-peer architecture

In the client/server architecture, where one machine is effectively running the game, and every other machine is just a terminal and renders whatever the server tells it to render. This architecture shows the same game in every machine since all the processing is done on one machine called the client/server. This encourages multi-player gaming.

Scripting

Scripting is not the same as AI. Scripting is where the developer takes full control over the scene setting up events that player has almost no involvement. This is very prominent in recent big games

where you have scripting in the form of story telling and pre-scripted flashbacks. There are various forms of scripting. Text-based scripting consists of thread texts being displayed. Visual scripting is more prominent these days with good game engines now available in the market. With visual scripting, you can script using game characters, trace paths for characters to follow and define animations to be used, making your scripting more realistic.

Programming and AI

After rendering, artificial intelligence (AI) is the most important element in game development. One of the most crucial aspect of the game, AI, defines your fun experience during game play. The better the AI, the more involving is the game play. AI is nothing but intelligent codes written in programming languages to take decisions based on your actions in the game. All non player characters (NPC) in the game are controlled by AI. Each NPC should have awareness of the surrounding world, that is, exact information of game objects around it and their behaviour. The programmer decides the extent of the knowledge that NPC can have about the world. AI consists of all decision making routines that determine how the NPCs in your game will behave and interact with the player.

World navigation (path finding) is definitely interesting for budding game developers. World navigation takes into account the complete geography and also the position of objects in the game world and then decides how an NPC navigates from one point to another. All possible options and corner cases have to be considered while writing the code for AI. That's what will make your game more interesting and exciting.

Now that you know the basic architecture of the game engine, you can try and twiddle different game engines and find what suits your need the best. There are a pretty decent number of free and open source game engines available. Some of the good ones we suggest are Unity, OGRE (it's a rendering engine only), PANDA 3D, SAGE. Almost every major gaming company has their own engine with which they create their games. 

